

中心極限定理のサイコロによる例示

著者：関勝寿 公開日：2024年8月30日 ソース：<https://sekika.github.io/2024/08/30/dice/>

中心極限定理によると、独立同分布の確率変数の和（あるいは平均）は、項数が大きくなると正規分布に近づく。ここではサイコロの目を例に説明する。

1つのサイコロを振ると、出る目は1から6までの整数であり、それぞれの目が出る確率は等しい。ここでは同様に確からしい一様分布を仮定する。

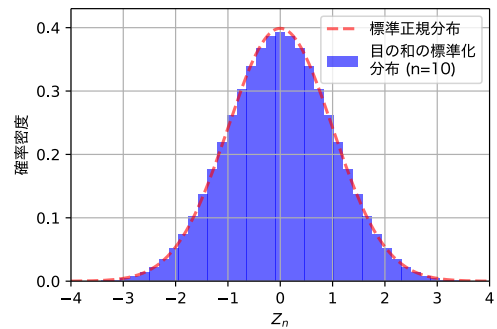
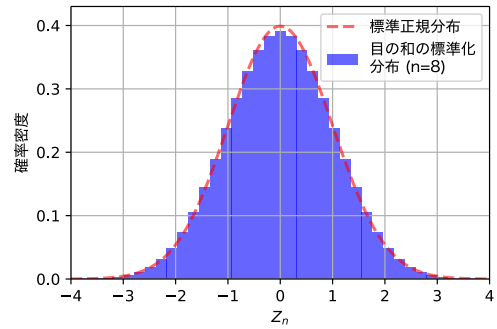
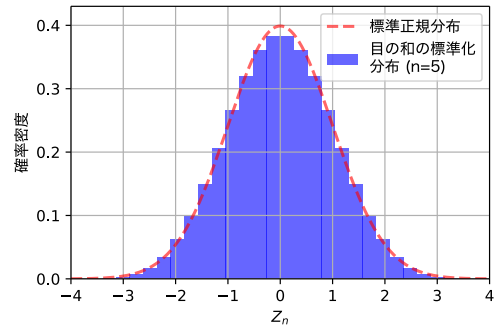
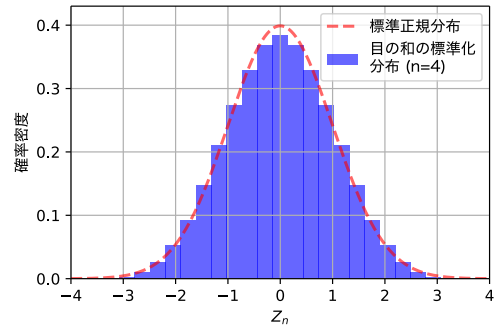
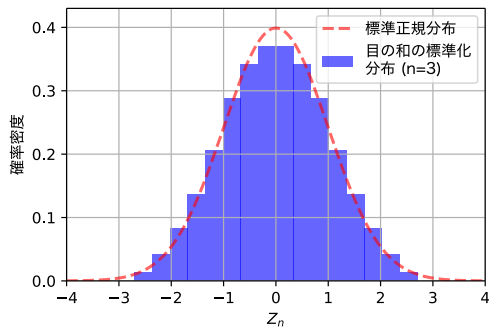
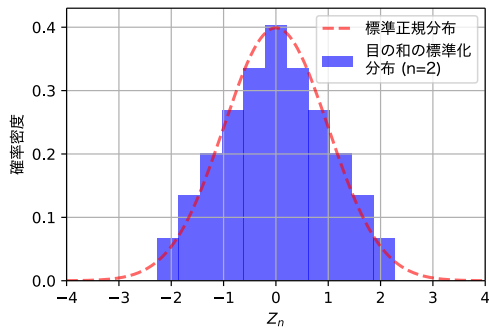
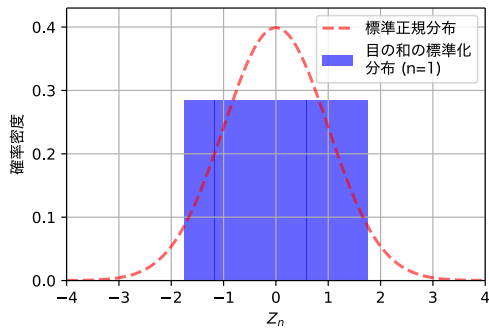
サイコロを n 回振ったとき、それぞれの目を $X_1, X_2, \dots, X_n$ と書く。期待値は $\mu = 3.5$ 、標準偏差は $\sigma \approx 1.71$ である。

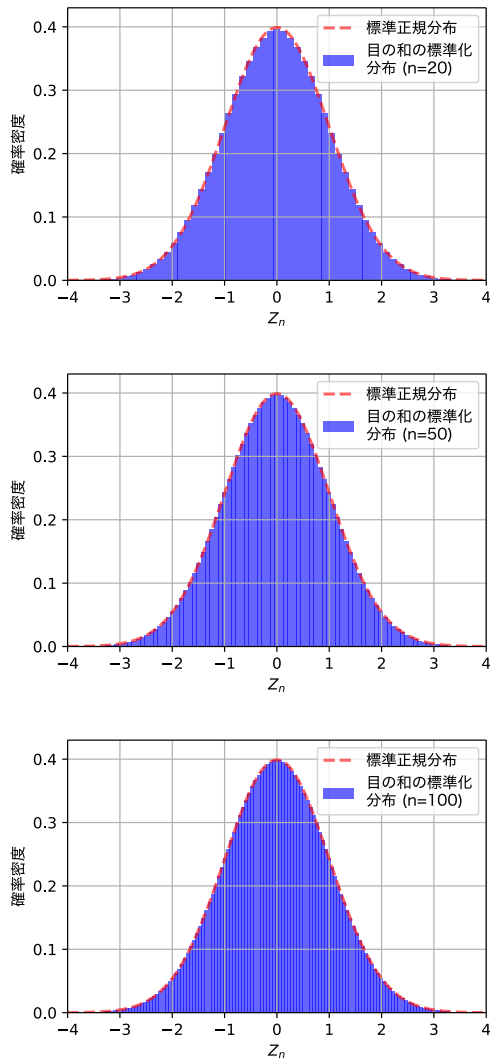
$$S_n = X_1 + X_2 + \dots + X_n$$

この和は平均 $n\mu$ 、標準偏差 $\sigma\sqrt{n}$ となる。標準化した変数は次式で表される。

$$Z_n = \frac{S_n - n\mu}{\sigma\sqrt{n}}$$

$n \rightarrow \infty$ において、 Z_n は標準正規分布 $N(0, 1)$ に収束する。以下は各 n における標準化分布と標準正規分布の比較である。





このように、 n が大きくなるにつれて標準正規分布に近づくことがわかる。図は乱数シミュレーションではなく、理論上の確率分布から作成している。

1. プログラム

この図を作成する Python プログラムを以下に示す。

```
#!/usr/bin/env python3
#
# 中心極限定理のサイコロによる例示
#
# サイコロを  $n$  回振ったときの目の和の確率の標準化分布が標準正規分布に漸近することを示す。
#
# Author: Katsutoshi Seki
# License: MIT License
#
# 日本語フォントのパスを設定
# 詳しくは https://sekika.github.io/2023/03/11/pyplot-japanese/
FONT_PATH = '/path/to/fontfile/HiraKakuProN-W4-AlphaNum-02.otf'

def main():
    import argparse
    parser = argparse.ArgumentParser(description=' 中心極限定理のサイコロによる例示 ')
    parser.add_argument('-e', '--ext', default='svg',
                        help=' 図のファイルの拡張子 (デフォルト svg) ')
    parser.add_argument('-b', '--black', action='store_true',
```

```
                        help=' 白黒の図を生成 ')
    args = parser.parse_args()
    for n in (1, 2, 3, 4, 5, 6, 8, 10, 20, 30, 40, 50, 100):
        dice_roll(n, args)

def dice_roll(n, args):
    import os
    import numpy as np
    import matplotlib.pyplot as plt
    from scipy.stats import norm
    from matplotlib.font_manager import FontProperties
    # サイコロの目の平均と標準偏差を計算
    mu = 3.5
    sigma = np.sqrt(np.mean((np.arange(1, 7) - mu)**2))

    # サイコロの目の和の理論的な確率分布を計算
    x = np.arange(n, 6 * n + 1)
    prob = np.zeros(len(x))
    memo = {}

    def dice_sum_prob(n, total):
        if (n, total) in memo:
            return memo[(n, total)]
        if n == 0:
            return 1 if total == 0 else 0
        if total < 0 or total > 6 * n:
            return 0
        result = sum(dice_sum_prob(n - 1, total - i) for i in range(1, 7))
        memo[(n, total)] = result
        return result
    for i in range(len(x)):
        prob[i] = dice_sum_prob(n, x[i])

    # 標準化
    mu_sum = n * mu
    sigma_sum = np.sqrt(n) * sigma
    z = (x - mu_sum) / sigma_sum

    # 標準正規分布の理論曲線
    x_norm = np.linspace(-4, 4, 1000)
    y_norm = norm.pdf(x_norm, 0, 1)

    # プロット開始
    if args.black:
        color = ['grey', 'black']
    else:
        color = ['blue', 'red']
    plt.figure(figsize=(4.5, 3))
    plt.subplots_adjust(left=0.13, right=0.98, top=0.98, bottom=0.15)
    fp = FontProperties(fname=os.path.expanduser(FONT_PATH))
    plt.plot(x_norm, y_norm,
             color=color[1], linestyle="dashed",
             alpha=0.6, linewidth=2, label=" 標準正規分布 ")

    # 確率をバーの幅で割った確率密度をバーの高さとする。すなわちバーの面積が
    # 確率となる。
    width = z[1] - z[0]
    plt.bar(z, prob / (6 ** n) / width, width=width,
            color=color[0], alpha=0.6, label=f" 目の和の標準化 \n 分布 (n={n})")
    plt.axis([-4, 4, 0, 0.43])

    plt.xlabel("$Z_n$")
    plt.ylabel(" 確率密度 ", fontproperties=fp)
    plt.legend(prop=fp)
    plt.grid(True)
    plt.savefig(f'dice{n}.{args.ext}')

if __name__ == "__main__":
    main()
```